BSCS – 5th Semester | PUACP | Web Technologies – EC-331

Topic: Server Configuration

By Noorayy Jam

Introduction

Server configuration is the process of setting up, managing, and optimizing a web server so that it can effectively host websites, handle client requests, and deliver web content securely and efficiently. It defines how the server communicates with clients, where files are stored, and how access, security, and performance are maintained.

A properly configured server ensures: - Fast and reliable content delivery
- Secure data communication
- Proper access control and permissions
- Efficient handling of multiple users and domains

Objectives of Server Configuration

1. Define **server behavior** (e.g., ports, directories, protocols).
2. Manage **multiple domains** using virtual hosting.
3. Configure **security settings**, SSL, and firewalls.
4. Enable **server logging** for monitoring and debugging.
5. Improve **speed and scalability** through caching and load balancing.

Common Types of Web Servers

1. **Apache HTTP Server** – Open-source, highly customizable, supports multiple modules.
2. **Nginx** – Lightweight, fast, and efficient for handling concurrent connections.
3. **Microsoft IIS (Internet Information Services)** – GUI-based server for Windows.
4. **LiteSpeed Server** – High-performance commercial server compatible with Apache configs.
5. **Tomcat Server** – Specially designed for Java-based web apps using JSP and Servlets.

Core Components of Server Configuration

1. Server Root Directory

The base directory that contains configuration files, website content, and logs.

```
Linux Path: /etc/httpd/ or /etc/apache2/
```

Contains: - `conf/` → configuration files (httpd.conf, apache2.conf) - `logs/` → access/error logs - `htdocs/` → main website files (HTML, CSS, JS)

2. Document Root

Defines the directory where website files are stored and served.

```
DocumentRoot "/var/www/html"
```

All HTML, CSS, JS, and PHP files reside here.

3. Port Configuration

Servers listen for incoming client requests on specific ports. - `80` → HTTP - `443` → HTTPS (secured using SSL/TLS) Example:

```
Listen 80
Listen 443
```

4. Server Configuration Files

- **httpd.conf / apache2.conf:** Main configuration files for Apache.
- Control everything from document root, ports, error handling, and directory permissions.

Virtual Hosting

Virtual hosting allows multiple websites to be hosted on a single physical server.

1. Name-Based Virtual Hosting

Uses domain names to identify sites (same IP address).

```
<VirtualHost *:80>
   ServerName www.site1.com
   DocumentRoot /var/www/site1
</VirtualHost>

<VirtualHost *:80>
   ServerName www.site2.com
```

```
   DocumentRoot /var/www/site2
</VirtualHost>
```

2. IP-Based Virtual Hosting

Each site is assigned a unique IP.

```
<VirtualHost 192.168.1.5:80>
   ServerName www.siteA.com
   DocumentRoot /var/www/siteA
</VirtualHost>
```

3. Port-Based Virtual Hosting

Sites are differentiated by port numbers.

```
<VirtualHost *:8080>
   ServerName www.blogsite.com
   DocumentRoot /var/www/blog
</VirtualHost>
```

Directory Configuration

Used to define permissions, options, and overrides for specific directories.

```
<Directory "/var/www/html">
   Options Indexes FollowSymLinks
   AllowOverride All
   Require all granted
</Directory>
```

- **Options:** Enables directory listing and symbolic links. - **AllowOverride:** Lets `.htaccess` override server rules. - **Require:** Grants access permissions.

.htaccess File (Directory-Level Configuration)

`.htaccess` allows per-directory configuration without changing the main config file. Common uses: - URL redirection - Authentication & access control - Custom error pages - Compression and caching

Example:

```
RewriteEngine On
RewriteRule ^oldpage.html$ newpage.html [R=301,L]
```

Server Modules

Modules extend the functionality of the server. - `mod_rewrite` → URL rewriting
- `mod_ssl` → Secure connections
- `mod_headers` → Modify HTTP headers
- `mod_proxy` → Reverse proxy and load balancing

Enable a module:

```
LoadModule rewrite_module modules/mod_rewrite.so
```

Logging and Monitoring

Logs are essential for debugging and performance monitoring. - **Access Log:** Records client requests.
Example: `/var/log/apache2/access.log` - **Error Log:** Records warnings, issues, or failed requests.
Example: `/var/log/apache2/error.log`

Benefits: - Tracks user activity
- Detects security breaches
- Helps troubleshoot server issues

Security Configuration

Security is critical for maintaining the integrity of the server. - **Disable Directory Listing:** Prevent users from browsing folders. - **Restrict Access by IP:**

```
<Directory "/admin">
   Require ip 192.168.1.10
</Directory>
```

- **Enable SSL/TLS:** Encrypts communication. - **Regular Updates:** Patch known vulnerabilities. - **Use Firewalls & Rate-Limiting:** To prevent DDoS attacks.

Performance Optimization

Caching

Stores frequently used data to speed up responses.

Compression

Use Gzip/Deflate to reduce file size before sending.

Load Balancing

Distribute traffic among multiple servers for stability.

KeepAlive

Keeps connections open for multiple requests:

```
KeepAlive On
MaxKeepAliveRequests 100
KeepAliveTimeout 5
```

Common Apache Commands

Command	Description
`sudo service apache2 start`	Start Apache server
`sudo service apache2 stop`	Stop Apache server
`sudo service apache2 restart`	Restart server
`apachectl configtest`	Test config syntax
`sudo tail -f /var/log/apache2/error.log`	Monitor errors in real-time

Sample Configuration Example

```
<VirtualHost *:80>
   ServerAdmin admin@example.com
   ServerName www.example.com
   DocumentRoot /var/www/example
   ErrorLog /var/log/apache2/example-error.log
   CustomLog /var/log/apache2/example-access.log combined
```

```
    <Directory "/var/www/example">
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Advanced Configurations

- **Environment Variables:** Control runtime behavior.
- **Proxy Configuration:** Enables load balancing and reverse proxy setups.
- **Security Headers:** Add HTTP headers for protection.

```
Header always set X-Frame-Options "DENY"
Header always set X-Content-Type-Options "nosniff"
```

Conclusion

Server configuration forms the backbone of every web application's performance and reliability. Properly configured servers: - Enhance website speed and scalability
- Provide strong security and access control
- Support multiple sites and environments efficiently
- Simplify administration and monitoring tasks

A well-managed server ensures a stable, secure, and optimized digital environment for both developers and end users.